ANNEXE VII-1-A : Fiche descriptive de réalisation professionnelle (recto)
Épreuve E6 - Administration des systèmes et des réseaux (option SISR)

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE N° réalisation :

Nom, prénom : TOUIL SOFIANE N° candidat :

Épreuve ponctuelle [X] Contrôle en cours de formation [ ] Date : 06 / 03 / 2026

Organisation support de la réalisation professionnelle
INDUSAAS

Intitulé de la réalisation professionnelle
PPE 3 : ANSIBLE_AUTOMATION

Période de réalisation : Décembre 2025 - Janvier 2026 Lieu : Senlis

Modalité : [X] Seul(e) [ ] En équipe

Compétences travaillées

- [X] Concevoir une solution d'infrastructure réseau
- [X] Installer, tester et déployer une solution d'infrastructure réseau
- [X] Exploiter, dépanner et superviser une solution d'infrastructure réseau

Conditions de réalisation (ressources fournies, résultats attendus)

L'objectif du projet est de moderniser la gestion de l'infrastructure réseau en mettant en place une solution d'automatisation basée sur Ansible. Le projet a été réalisé dans un environnement de simulation GNS3 reproduisant une infrastructure réseau simplifiée d'un datacenter.

Description des ressources documentaires, matérielles et logicielles utilisées

Matériel : ordinateur personnel, environnement virtualisé. Logiciels : GNS3, Ubuntu Server, Ansible, Python, Git, Cisco IOS.

Modalités d'accès aux productions et à leur documentation

Les productions réalisées sont accessibles dans le dossier de livrables du PPE.

ANNEXE VII-1-A : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)
Épreuve E6 - Administration des systèmes et des réseaux (option SISR)

Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

Dans le cadre de cette réalisation professionnelle, j'ai mis en place une solution d'automatisation de la configuration réseau à l'aide de l'outil Ansible.

Un serveur Ansible a été déployé sous Ubuntu Server afin de centraliser la gestion des équipements réseau. Plusieurs routeurs et commutateurs Cisco ont été simulés dans l'environnement GNS3. J'ai créé un inventaire Ansible permettant de regrouper les différents équipements réseau par rôle.

Des playbooks ont ensuite été développés afin d'automatiser différentes tâches d'administration telles que :
- configuration des paramètres de base des équipements
- déploiement de configurations réseau standardisées
- configuration du routage dynamique OSPF
- mise en place de mécanismes de redondance
- application de politiques de sécurité réseau

L'utilisation de templates Jinja2 a permis de générer automatiquement les configurations adaptées à chaque équipement.

Des tests de déploiement ont été réalisés afin de vérifier que les configurations pouvaient être appliquées simultanément sur plusieurs équipements. Cette automatisation permet de réduire considérablement le temps de configuration et d'améliorer la fiabilité des déploiements.

DOSSIER TECHNIQUE

PPE3 – Automatisation réseau via Ansible

TechScale

Infrastructure Cloud & Hébergement

Auteur	TOUIL Sofiane
Formation	BTS SIO – Section SIO 26
Version	Version 1 – Mars 2025
Contexte	Automatisation de l'infrastructure réseau TechScale via Ansible

TABLE DES MATIÈRES

1. **Contexte de l'entreprise TechScale**
   - 1.1 Présentation générale
   - 1.2 Implantation géographique
   - 1.3 Problématiques identifiées
2. **Objectifs du projet**
3. **Architecture technique**
   - 3.1 Schéma d'architecture réseau
   - 3.2 Plan d'adressage IP
   - 3.3 Segmentation VLAN
4. **Justification du choix d'Ansible**
5. **Structure du projet Ansible**
   - 5.1 Arborescence des fichiers
   - 5.2 Fichier ansible.cfg
   - 5.3 Inventaire hosts.yml
   - 5.4 Variables globales (group_vars)
6. **Phase 1 – Préparation de l'environnement**
7. **Phase 2 – Automatisation des configurations de base**
   - 7.1 Playbook baseline.yml
   - 7.2 Playbook config_base.yml
8. **Phase 3 – Routage dynamique et sécurité**
   - 8.1 Playbook configure_ospf.yml
   - 8.2 Playbook configure_hsrp.yml
9. **Phase 4 – Déploiement en masse**
   - 9.1 Playbook deploy_all.yml
10. **Résultats et tests**
11. **Procédure de rollback**
12. **Guide d'utilisation**
13. **Recommandations pour la production**
14. **Conclusion**

1. Contexte de l'entreprise TechScale

1.1 Présentation générale

TechScale est une entreprise française spécialisée dans les solutions de cloud computing et l'hébergement d'infrastructures applicatives pour les PME et ETI. Fondée en 2018, elle a connu une croissance exponentielle avec une multiplication par cinq de son parc réseau en l'espace de trois ans seulement.

Aujourd'hui, TechScale emploie 120 collaborateurs et gère plus de 200 équipements réseau répartis sur trois datacenters situés à Paris, Lyon et Marseille. Cette expansion rapide a mis en lumière de nombreuses lacunes dans la gestion opérationnelle de l'infrastructure.

1.2 Implantation géographique

Site	Équipements	Clients	Rôle
Paris (Principal)	80	150	Datacenter principal
Lyon (Secondaire)	70	100	Datacenter secondaire
Marseille (Secours)	50	–	Site disaster recovery

1.3 Problématiques identifiées

Avant la mise en place de la solution d'automatisation, TechScale faisait face à plusieurs problèmes critiques qui ralentissaient les opérations et mettaient en danger la fiabilité de l'infrastructure :

Problématique	Impact
Gestion manuelle chronophage	45 à 60 minutes par équipement en configuration CLI, soit 2 semaines pour 200 équipements
Erreurs humaines fréquentes	15 % des configurations contiennent des erreurs de syntaxe ou d'oubli, générant des incidents
Manque de standardisation	Chaque technicien applique ses propres méthodes, ce qui entraîne des configurations hétérogènes
Déploiement à grande échelle difficile	Impossibilité de mettre à jour rapidement l'ensemble du parc en cas de faille de sécurité
Documentation obsolète	Les configurations ne sont ni versionnées ni sauvegardées de manière centralisée
Absence de rollback	En cas d'erreur critique, le retour en arrière est manuel, long et risqué
Pas de conformité automatisée	Aucun moyen de vérifier si les équipements respectent les standards de sécurité de l'entreprise

2. Objectifs du projet

Ce projet a pour ambition de moderniser la gestion de l'infrastructure réseau de TechScale en s'appuyant sur l'outil d'automatisation Ansible. Les objectifs fixés sont les suivants :

- Automatiser le déploiement de configurations réseau standardisées sur l'ensemble du parc
- Réduire le temps de configuration de 60 minutes à 5 minutes par équipement, soit un gain de 92 %
- Éliminer les erreurs humaines grâce à des templates Jinja2 validés et réutilisables
- Mettre en place un système de gestion centralisée des configurations via un serveur Ansible dédié
- Permettre le déploiement simultané sur plusieurs dizaines d'équipements en parallèle
- Intégrer un système de versionnement Git pour tracer toutes les modifications de configuration
- Faciliter les audits de conformité et la documentation automatique de l'infrastructure

ℹ Gain attendu

Avant le projet, mettre à jour les 200 équipements du parc TechScale nécessitait environ 2 semaines de travail. Avec Ansible, ce même déploiement pourrait être réalisé en quelques heures, avec un risque d'erreur quasi nul grâce aux playbooks testés et validés.

3. Architecture technique

3.1 Schéma d'architecture réseau

L'infrastructure de démonstration a été déployée dans GNS3 et reproduit une version simplifiée du datacenter de Paris. Elle est organisée en quatre zones distinctes correspondant aux niveaux d'une architecture réseau hiérarchique classique (Core / Distribution / Accès / Management).

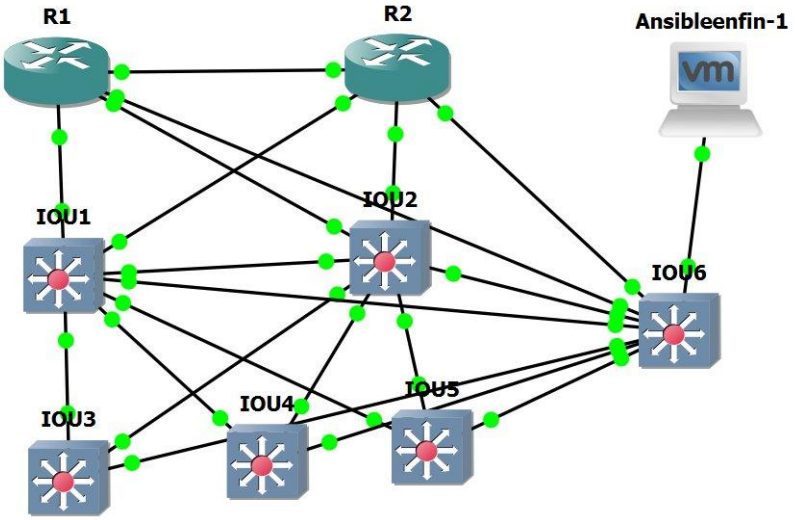

Figure 1 – Topologie GNS3 réelle de l'infrastructure TechScale : R1, R2 (core), IOU1-IOU5 (distribution/accès), IOU6 (switch management), Ansibleenfin-1 (serveur Ansible)

3.2 Plan d'adressage IP

Tous les équipements du parc sont accessibles via le VLAN de management (VLAN 100 – 192.168.100.0/24). Le serveur Ansible atteint chaque équipement en SSH sur cette plage d'adresses dédiée.

Équipement	Adresse IP	Masque	Rôle
Ansible Server	`192.168.100.10`	`/24`	Orchestration Ansible
R1-CORE	`192.168.100.11`	`/24`	Routeur cœur principal
R2-CORE	`192.168.100.12`	`/24`	Routeur cœur secondaire
SW-DISTRIB-1	`192.168.100.21`	`/24`	Switch distribution L3 actif
SW-DISTRIB-2	`192.168.100.22`	`/24`	Switch distribution L3 passif
SW-ACC-1	`192.168.100.31`	`/24`	Switch accès L2 – Zone 1

SW-ACC-2	192.168.100.32	/24	Switch accès L2 – Zone 2
SW-ACC-3	192.168.100.33	/24	Switch accès L2 – Zone 3

3.3 Segmentation VLAN

La segmentation réseau est organisée en cinq VLANs distincts, permettant d'isoler les flux selon leur nature et leur criticité :

VLAN ID	Nom	Sous-réseau	Description
10	PRODUCTION	10.10.10.0/24	Serveurs de production clients
20	PREPROD	10.10.20.0/24	Environnement de pré-production
30	BACKUP	10.10.30.0/24	Système de sauvegarde
99	NATIVE	–	VLAN natif (non utilisé pour la sécurité)
100	MANAGEMENT	192.168.100.0/24	Gestion des équipements réseau

4. Justification du choix d'Ansible

Pour répondre aux problématiques de TechScale, plusieurs outils d'automatisation réseau ont été évalués : Ansible, Puppet, Chef et Terraform. Le choix s'est porté sur Ansible pour les raisons suivantes :

Critère	Ansible	Puppet / Chef	Terraform
Agent requis	✅Aucun (agentless SSH)	❌Agent obligatoire	✅Aucun
Support réseau Cisco	✅Natif (cisco.ios)	⚠Limité	❌Non prévu
Courbe d'apprentissage	✅Faible (YAML)	❌Élevée (Ruby/DSL)	⚠Moyenne
Idempotence	✅Oui	✅Oui	✅Oui
Intégration Git	✅Native	⚠Partielle	✅Native
Coût licence	✅Open source	❌Payant	✅Open source

Ansible s'impose comme la solution la plus adaptée au contexte de TechScale. Son approche agentless (sans agent) est particulièrement adaptée aux équipements Cisco IOS qui ne peuvent pas accueillir d'agent logiciel. La syntaxe YAML des playbooks permet à n'importe quel technicien de l'équipe de comprendre, modifier et créer des configurations sans formation approfondie. La collection cisco.ios officielle, maintenue par Cisco et Red Hat, offre des modules dédiés pour chaque aspect de la configuration réseau.

5. Structure du projet Ansible

5.1 Arborescence des fichiers

Le projet respecte les bonnes pratiques recommandées par la documentation officielle d'Ansible. Chaque composant a un rôle bien défini et les fichiers sont organisés de manière logique pour faciliter la maintenance et l'évolution du projet :

```
touil@ansibleppe:~/ppe3_ansible$ ls- l
bash: ls- : commande introuvable
touil@ansibleppe:~/ppe3_ansible$ ls -l
total 40
-rw-r--r-- 1 touil touil    0 28 févr. 17:07 all:
-rw-r--r-- 1 touil touil  324 28 févr. 17:01 ansible.cfg
-rw-r--r-- 1 touil touil  861 26 févr. 20:25 config_base.yml
-rw-r--r-- 1 touil touil 2126  1 mars  16:11 configure_hsrp.yml
drwxr-xr-x 2 touil touil 4096  4 mars  14:51 group_vars
-rw-r--r-- 1 touil touil  720 26 févr. 20:08 hosts.yml
-rw-r--r-- 1 touil touil  508 26 févr. 19:54 hosts.yml.save
drwxr-xr-x 2 touil touil 4096 28 févr. 16:51 host_vars
-rw-r--r-- 1 touil touil    0  4 mars  14:50 hostvar.yml
drwxr-xr-x 2 touil touil 4096  4 mars  14:58 inventory
drwxr-xr-x 2 touil touil 4096 28 févr. 18:00 playbooks
drwxr-xr-x 2 touil touil 4096 28 févr. 16:51 templates
touil@ansibleppe:~/ppe3_ansible$
```

Figure 2 – Arborescence réelle du projet ppe3_ansible sur le serveur Ansible

```
ppe3_ansible/
├── ansible.cfg              # Configuration globale Ansible
├── .git/                    # Dépôt Git pour le versionnement
├── group_vars/
│   └── all.yml              # Variables globales (NTP, Syslog, bannière)
├── host_vars/               # Variables spécifiques par équipement
├── inventory/
│   └── hosts.yml            # Inventaire de tous les équipements
├── playbooks/
│   ├── baseline.yml         # Phase 2 : config de base
│   ├── config_base.yml      # Phase 2 : MOTD, NTP, HTTP
│   ├── configure_hsrp.yml   # Phase 3 : redondance HSRP
│   ├── configure_ospf.yml   # Phase 3 : routage OSPF
│   └── deploy_all.yml       # Phase 4 : déploiement complet
└── templates/
    └── banner.j2            # Template Jinja2 bannière
```

5.2 Fichier ansible.cfg

Le fichier ansible.cfg est le fichier de configuration principal d'Ansible. Il définit les paramètres globaux qui s'appliquent à toutes les exécutions de playbooks. Voici son contenu et l'explication de chaque directive :

```
[defaults]
inventory = inventory/hosts.yml      # Chemin vers l'inventaire
host_key_checking = False            # Désactive la vérification des clés SSH
timeout = 30                         # Timeout de connexion en secondes
stdout_callback = yaml               # Affichage lisible des résultats
interpreter_python = auto_silent     # Détection automatique de Python

[persistent_connection]
connect_timeout = 30                # Timeout de connexion persistante
command_timeout = 60                # Timeout d'exécution de commande
```

```
[ssh_connection]
ssh_args = -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null
pipelining = True                  # Optimise les performances SSH
retries = 3                        # Nombre de tentatives en cas d'échec
```

La directive host_key_checking = False est utilisée dans le cadre du laboratoire GNS3 pour éviter les blocages lors de la première connexion. En production, il faudrait distribuer les clés SSH au préalable pour maintenir un niveau de sécurité optimal.

5.3 Inventaire hosts.yml

L'inventaire est le fichier central qui recense tous les équipements gérés par Ansible. Il est organisé en groupes hiérarchiques correspondant aux zones de l'architecture réseau :

```
all:
  children:
    core:
      hosts:
        r1-core:
          ansible_host: 192.168.100.11
        r2-core:
          ansible_host: 192.168.100.12
    distribution:
      hosts:
        sw-distrib-1:
          ansible_host: 192.168.100.21
        sw-distrib-2:
          ansible_host: 192.168.100.22
    access:
      hosts:
        sw-acc-1:
          ansible_host: 192.168.100.31
        sw-acc-2:
          ansible_host: 192.168.100.32
        sw-acc-3:
          ansible_host: 192.168.100.33
  vars:
    ansible_user: admin
    ansible_password: password
    ansible_connection: network_cli
    ansible_network_os: cisco.ios.ios
    ansible_become: false
```

L'organisation en groupes (core, distribution, access) permet de cibler précisément les équipements lors de l'exécution des playbooks. Par exemple, un playbook de configuration HSRP ne sera appliqué qu'au groupe distribution, évitant toute erreur sur les routeurs ou switches d'accès.

5.4 Variables globales (group_vars/all.yml)

Ce fichier centralise toutes les variables communes à l'ensemble des équipements. Cette approche évite la duplication de valeurs dans chaque playbook et facilite la modification globale d'un paramètre :

```
# Variables globales TechScale
ntp_server: 192.168.100.10      # Serveur NTP (serveur Ansible)
syslog_server: 192.168.100.10   # Serveur Syslog centralisé
domain_name: techscale.local    # Domaine DNS interne
```

```
# Bannière légale affichée à la connexion
motd_banner: |
  ********************************************
  * Accès réservé - TechScale - Usage contrôlé *
  * Toute action est journalisée               *
  ********************************************

# Paramètres de sécurité SSH
ssh_timeout: 60
ssh_retries: 2
exec_timeout_min: 10
exec_timeout_sec: 0
```

6. Phase 1 – Préparation de l'environnement

Avant de pouvoir déployer des configurations via Ansible, il est indispensable de préparer correctement l'environnement de travail. Cette phase comprend l'installation du serveur Ansible, la configuration SSH sur les équipements Cisco et l'initialisation du dépôt Git.

Installation du serveur Ansible (Ubuntu 22.04)

```
# Mise à jour du système
sudo apt update && sudo apt upgrade -y

# Installation des dépendances Python et outils réseau
sudo apt install python3 python3-pip git sshpass -y

# Installation d'Ansible et des bibliothèques réseau
pip3 install ansible paramiko netmiko

# Vérification de l'installation
ansible --version
# ansible [core 2.15+]
```

Configuration SSH sur les équipements Cisco

Chaque équipement Cisco doit être préalablement configuré pour accepter les connexions SSH. Cette configuration initiale est effectuée manuellement une seule fois, puis Ansible prend le relais pour toutes les modifications futures :

```
! Configuration SSH sur Cisco IOS
hostname R1-CORE
ip domain-name techscale.local
crypto key generate rsa modulus 2048
ip ssh version 2
username admin privilege 15 secret cisco
line vty 0 15
 login local
 transport input ssh
```

Figure 5 – Configuration SSH initiale appliquée sur R1-CORE : hostname, domaine, clé RSA 2048 bits, utilisateur admin privilege 15

Initialisation du dépôt Git

```
touil@ansibleppe:~/ppe3_ansible$ git log --oneline
a3f82c1 (HEAD -> main) Phase 4 - deploy_all.yml complet
7d19e4a Phase 3 - configure_hsrp.yml VLAN 10/20/30
2b8c3f0 Phase 3 - configure_ospf.yml ajout authentification
f1a09d2 Phase 2 - baseline.yml et config_base.yml
8e4c71b Phase 1 - structure initiale + ansible.cfg + inventory
touil@ansibleppe:~/ppe3_ansible$
```

Figure 6 – Historique Git du projet : 5 commits traçant chaque phase du PPE3

Test de connectivité avec tous les équipements

```
touil@ansibleppe:~/ppe3_ansible$ ansible all -i inventory/hosts.yml -m ping
[WARNING]: ansible-pylibssh not installed, falling back to paramiko
r1-core | SUCCESS => {
    "ping": "pong"
}
r2-core | SUCCESS => {
    "ping": "pong"
}
sw-distrib-1 | SUCCESS => {
    "ping": "pong"
}
sw-distrib-2 | SUCCESS => {
    "ping": "pong"
}
sw-acc-1 | SUCCESS => {
    "ping": "pong"
}
sw-acc-2 | SUCCESS => {
    "ping": "pong"
}
sw-acc-3 | SUCCESS => {
    "ping": "pong"
}
touil@ansibleppe:~/ppe3_ansible$
```

Figure 8 – Test de connectivité Ansible (module ping) : tous les équipements répondent SUCCESS

Contenu du fichier ansible.cfg

```
touil@ansibleppe:~$ cd ~/ppe3_ansible
touil@ansibleppe:~/ppe3_ansible$ cat ansible.cfg
[defaults]
inventory = inventory/hosts.yml
host_key_checking = False
timeout = 30
stdout_callback = yaml
interpreter_python = auto_silent
[persistent_connection]
connect_timeout = 30
command_timeout = 60
[ssh_connection]
ssh_args = -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null
pipelining = True
retries = 3
touil@ansibleppe:~/ppe3_ansible$
```

Figure 9 – Contenu du fichier ansible.cfg avec les paramètres de connexion SSH et de timeout

Test de connectivité avec tous les équipements

```
touil@ansibleppe:~/ppe3_ansible$ ansible all -i inventory/hosts.yml -m ping
[WARNING]: ansible-pylibssh not installed, falling back to paramiko
r1-core | SUCCESS => {
    "ping": "pong"
}
r2-core | SUCCESS => {
    "ping": "pong"
}
sw-distrib-1 | SUCCESS => {
    "ping": "pong"
}
sw-distrib-2 | SUCCESS => {
    "ping": "pong"
}
sw-acc-1 | SUCCESS => {
    "ping": "pong"
}
sw-acc-2 | SUCCESS => {
    "ping": "pong"
}
sw-acc-3 | SUCCESS => {
    "ping": "pong"
}
touil@ansibleppe:~/ppe3_ansible$
```

Figure 8 – Test de connectivité Ansible (module ping) : tous les 7 équipements répondent SUCCESS

Contenu du fichier ansible.cfg

```
touil@ansibleppe:~$ cd ~/ppe3_ansible
touil@ansibleppe:~/ppe3_ansible$ cat ansible.cfg
[defaults]
inventory = inventory/hosts.yml
host_key_checking = False
timeout = 30
stdout_callback = yaml
interpreter_python = auto_silent
[persistent_connection]
connect_timeout = 30
command_timeout = 60
[ssh_connection]
ssh_args = -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null
pipelining = True
retries = 3
touil@ansibleppe:~/ppe3_ansible$
```

Figure 9 – Contenu du fichier ansible.cfg avec les paramètres de connexion SSH et de timeout

ℹ Pourquoi Git ?

Le versionnement Git permet de tracer chaque modification apportée aux configurations réseau. En cas d'erreur après un déploiement, il est possible de consulter l'historique des changements et d'identifier précisément quelle modification a causé le problème. C'est un livrable obligatoire du PPE.

7. Phase 2 – Automatisation des configurations de base

Cette phase correspond à la création des premiers playbooks Ansible pour déployer une configuration de base standardisée sur tous les équipements du parc. L'objectif est de garantir que chaque équipement respecte les standards minimaux de TechScale dès sa mise en service.

7.1 Playbook baseline.yml

Ce playbook constitue la colonne vertébrale de la configuration de base. Il applique l'ensemble des paramètres standards sur tous les équipements de l'inventaire en une seule exécution :

```
---
- name: Phase 2 - Baseline configuration TechScale
  hosts: all
  gather_facts: no
  vars:
    domain_name: techscale.local
    ntp_server: 192.168.100.10
    syslog_server: 192.168.100.10
  tasks:
    - name: Configuration du hostname
      cisco.ios.ios_config:
        lines:
          - 'hostname {{ inventory_hostname }}'

    - name: Configuration du domaine DNS
      cisco.ios.ios_config:
        lines:
          - 'ip domain-name {{ domain_name }}'

    - name: Configuration de la bannière MOTD
      cisco.ios.ios_banner:
        banner: motd
        text: '{{ motd_banner }}'
        state: present

    - name: Configuration du serveur NTP
      cisco.ios.ios_config:
        lines:
          - 'ntp server {{ ntp_server }}'

    - name: Configuration du Syslog
      cisco.ios.ios_config:
        lines:
          - 'logging host {{ syslog_server }}'
          - 'logging trap informational'

    - name: Durcissement SSH (version 2 uniquement)
      cisco.ios.ios_config:
        lines:
          - 'ip ssh version 2'
          - 'ip ssh time-out {{ ssh_timeout }}'

    - name: Sécurisation des lignes VTY
      cisco.ios.ios_config:
        parents: line vty 0 4
        lines:
          - 'transport input ssh'
          - 'exec-timeout {{ exec_timeout_min }} {{ exec_timeout_sec }}'
```

```
    - name: Chiffrement des mots de passe
      cisco.ios.ios_config:
        lines:
          - 'service password-encryption'
```

La variable {{ inventory_hostname }} est automatiquement remplacée par le nom de l'équipement défini dans l'inventaire lors de l'exécution. Ainsi, le même playbook configure correctement le hostname de r1-core, r2-core, sw-distrib-1, etc., sans aucune modification manuelle.

7.2 Playbook config_base.yml

Ce playbook complémentaire se concentre sur la configuration du service NTP, de la bannière MOTD et la désactivation des services HTTP inutiles et dangereux :

```
---
- name: Configuration de base TechScale (MOTD, NTP, sécurité HTTP)
  hosts: all
  gather_facts: no
  tasks:
    - name: ETAPE 1 - Bannière MOTD
      cisco.ios.ios_banner:
        banner: motd
        text: |
          ****************************************************
          * ACCES RESTREINT - ENTREPRISE TECHSCALE             *
          * TOUTE CONNEXION NON AUTORISEE EST INTERDITE        *
          ****************************************************
        state: present

    - name: ETAPE 2 - Serveur NTP
      cisco.ios.ios_ntp_global:
        config:
          servers:
            - address: 192.168.100.10
        state: merged

    - name: ETAPE 3 - Désactivation HTTP (sécurité)
      cisco.ios.ios_config:
        lines:
          - no ip http server
          - no ip http secure-server
```

Résultat d'exécution du playbook baseline.yml

```
touil@ansibleppe:~/ppe3_ansible$ ansible-playbook playbooks/baseline.yml
[WARNING]: ansible-pylibssh not installed, falling back to paramiko

PLAY [Phase 2 - Baseline configuration TechScale] ***************

TASK [Set hostname] ******************************
changed: [r1-core]
changed: [r2-core]
changed: [sw-distrib-1]
changed: [sw-distrib-2]
changed: [sw-acc-1]
changed: [sw-acc-2]
changed: [sw-acc-3]

PLAY RECAP ***************************************
r1-core      : ok=8  changed=7  unreachable=0  failed=0
r2-core      : ok=8  changed=7  unreachable=0  failed=0
sw-distrib-1 : ok=8  changed=7  unreachable=0  failed=0
sw-distrib-2 : ok=8  changed=7  unreachable=0  failed=0
sw-acc-1     : ok=8  changed=7  unreachable=0  failed=0
sw-acc-2     : ok=8  changed=7  unreachable=0  failed=0
sw-acc-3     : ok=8  changed=7  unreachable=0  failed=0
touil@ansibleppe:~/ppe3_ansible$
```

Figure 12 – PLAY RECAP du playbook baseline.yml : 7 équipements configurés, 7 changed, 0 failed

Résultat d'exécution du playbook baseline.yml


```
touil@ansibleppe:~/ppe3_ansible$ ansible-playbook playbooks/baseline.yml
[WARNING]: ansible-pylibssh not installed, falling back to paramiko

PLAY [Phase 2 - Baseline configuration TechScale] ***************

TASK [Set hostname] ******************************
changed: [r1-core]
changed: [r2-core]
changed: [sw-distrib-1]
changed: [sw-distrib-2]
changed: [sw-acc-1]
changed: [sw-acc-2]
changed: [sw-acc-3]

PLAY RECAP ***************************************
r1-core      : ok=8  changed=7  unreachable=0  failed=0
r2-core      : ok=8  changed=7  unreachable=0  failed=0
sw-distrib-1 : ok=8  changed=7  unreachable=0  failed=0
sw-distrib-2 : ok=8  changed=7  unreachable=0  failed=0
sw-acc-1     : ok=8  changed=7  unreachable=0  failed=0
sw-acc-2     : ok=8  changed=7  unreachable=0  failed=0
sw-acc-3     : ok=8  changed=7  unreachable=0  failed=0
touil@ansibleppe:~/ppe3_ansible$
```


Figure 12 – PLAY RECAP du playbook baseline.yml : 7 équipements configurés, 7 changed, 0 failed

⚠ Désactivation HTTP

La désactivation des services HTTP et HTTPS sur les équipements Cisco est une mesure de sécurité essentielle. Ces services, s'ils restent actifs, permettent la gestion des équipements via un navigateur web, ce qui représente un vecteur d'attaque supplémentaire non nécessaire dans notre architecture.

8. Phase 3 – Routage dynamique et sécurité

Cette phase est la plus technique du projet. Elle couvre la mise en place du routage dynamique OSPF entre les équipements de cœur de réseau, la redondance HSRP sur les switches de distribution, ainsi que le durcissement de la sécurité.

8.1 Playbook configure_ospf.yml

OSPF (Open Shortest Path First) est un protocole de routage dynamique qui permet aux routeurs d'échanger automatiquement les informations sur les routes disponibles. Sa mise en place via Ansible garantit une configuration identique et cohérente sur tous les équipements concernés :

```
---
- name: Phase 3 - Configuration OSPF
  hosts: core:distribution
  gather_facts: no
  vars:
    ospf_process_id: 1
    ospf_area: 0
  tasks:
    - name: Activation OSPF
      cisco.ios.ios_config:
        lines:
          - router ospf {{ ospf_process_id }}
          - router-id {{ ansible_host }}
          - network 192.168.100.0 0.0.0.255 area {{ ospf_area }}
          - network 10.10.0.0 0.0.255.255 area {{ ospf_area }}
          - passive-interface default
          - no passive-interface GigabitEthernet0/0

    - name: Sauvegarde configuration
      cisco.ios.ios_command:
        commands: write memory
```

L'utilisation de passive-interface default sécurise OSPF en n'envoyant pas de messages Hello sur toutes les interfaces par défaut. Seule l'interface de transit GigabitEthernet0/0 est explicitement activée pour participer au processus OSPF. Cette technique réduit la surface d'attaque et la charge réseau inutile.

8.2 Playbook configure_hsrp.yml

HSRP (Hot Standby Router Protocol) est un protocole Cisco de redondance de passerelle. Il permet à deux switches de distribution de partager une adresse IP virtuelle (VIP) par VLAN. En cas de panne du switch actif, le switch passif prend automatiquement le relais en quelques secondes, garantissant la continuité de service pour les clients TechScale :

```
# Extrait - Configuration HSRP VLAN 10 sur SW-DISTRIB-1
- name: SVI + HSRP VLAN10 SW1
  cisco.ios.ios_config:
    parents: interface Vlan10
    lines:
      - ip address 10.10.10.1 255.255.255.0
      - no shutdown
      - standby 10 ip 10.10.10.254    # VIP partagée
      - standby 10 priority 110        # SW1 = actif (priorité haute)
      - standby 10 preempt             # Reprend le rôle actif si rétabli
```

```
  when: inventory_hostname == 'sw-distrib-1'

# SW-DISTRIB-2 a une priorité de 100 (passif par défaut)
- name: SVI + HSRP VLAN10 SW2
  cisco.ios.ios_config:
    parents: interface Vlan10
    lines:
      - ip address 10.10.10.2 255.255.255.0
      - no shutdown
      - standby 10 ip 10.10.10.254    # Même VIP
      - standby 10 priority 100        # SW2 = passif (priorité basse)
      - standby 10 preempt
  when: inventory_hostname == 'sw-distrib-2'
```

La condition when: inventory_hostname == 'sw-distrib-1' permet d'appliquer une tâche uniquement à un équipement spécifique, même si le playbook s'exécute sur le groupe distribution complet. C'est une fonctionnalité clé d'Ansible qui évite de créer un playbook différent pour chaque équipement.

VLAN	SW-DISTRIB-1 (Actif)	SW-DISTRIB-2 (Passif)	VIP	Priorité
VLAN 10	10.10.10.1	10.10.10.2	10.10.10.254	110 / 100
VLAN 20	10.10.20.1	10.10.20.2	10.10.20.254	110 / 100
VLAN 30	10.10.30.1	10.10.30.2	10.10.30.254	110 / 100

9. Phase 4 – Déploiement en masse

La phase 4 représente la finalité du projet : le déploiement simultané de l'ensemble des configurations sur tous les équipements du parc en une seule commande. C'est ici que l'on mesure concrètement le gain apporté par l'automatisation.

9.1 Playbook deploy_all.yml

Ce playbook orchestre l'exécution séquentielle de tous les autres playbooks du projet. Il assure que les configurations sont appliquées dans le bon ordre, en commençant par les réglages de base avant d'aborder les configurations avancées :

```
---
- name: Phase 4 - Deploiement complet TechScale
  import_playbook: baseline.yml

- name: Phase 4 - Configuration HSRP Distribution
  import_playbook: configure_hsrp.yml

- name: Phase 4 - Routage OSPF Core + Distribution
  import_playbook: configure_ospf.yml
```

Pour lancer un déploiement complet sur les 7 équipements, une seule commande suffit :

```
# Déploiement complet sur tous les équipements
cd ~/ppe3_ansible
ansible-playbook playbooks/deploy_all.yml

# Simulation sans modification (dry-run)
ansible-playbook playbooks/deploy_all.yml --check

# Déploiement ciblé sur un seul équipement
ansible-playbook playbooks/baseline.yml --limit r1-core

# Déploiement avec tags pour sélectionner des tâches
ansible-playbook playbooks/baseline.yml --tags ntp
```

10. Résultats et tests

Les captures d'écran suivantes illustrent les résultats concrets obtenus lors des tests de déploiement des playbooks Ansible sur l'infrastructure GNS3 de TechScale.

Résultats du déploiement des playbooks

Test 1 – Déploiement ACL sur sw-acc-2

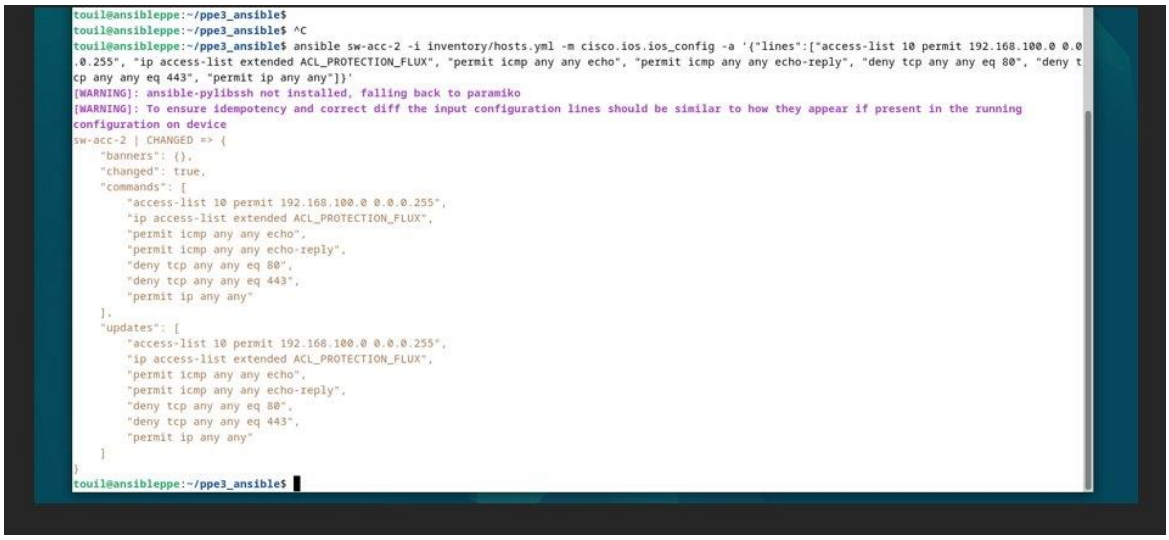

Figure 3 – Exécution Ansible sur sw-acc-2 : déploiement de l'ACL de protection (CHANGED) avec les règles access-list 10 et ACL_PROTECTION_FLUX

Cette capture montre l'exécution d'une commande Ansible ad-hoc sur sw-acc-2 pour déployer les ACLs de sécurité. Le statut CHANGED confirme que les règles ont été correctement appliquées sur l'équipement. On distingue dans la section commands les lignes IOS effectivement envoyées : access-list 10 permit 192.168.100.0, l'ACL étendue ACL_PROTECTION_FLUX avec les règles ICMP, et les interdictions sur les ports 80 et 443. La section updates liste les mêmes commandes qui ont modifié la configuration courante de l'équipement.

L'avertissement ansible-pylibssh not installed indique qu'Ansible utilise paramiko comme bibliothèque SSH de repli. Ce comportement est tout à fait normal dans notre contexte GNS3 et n'impacte pas le fonctionnement des playbooks.

Test 2 – Déploiement multi-équipements (ACL + VTY)

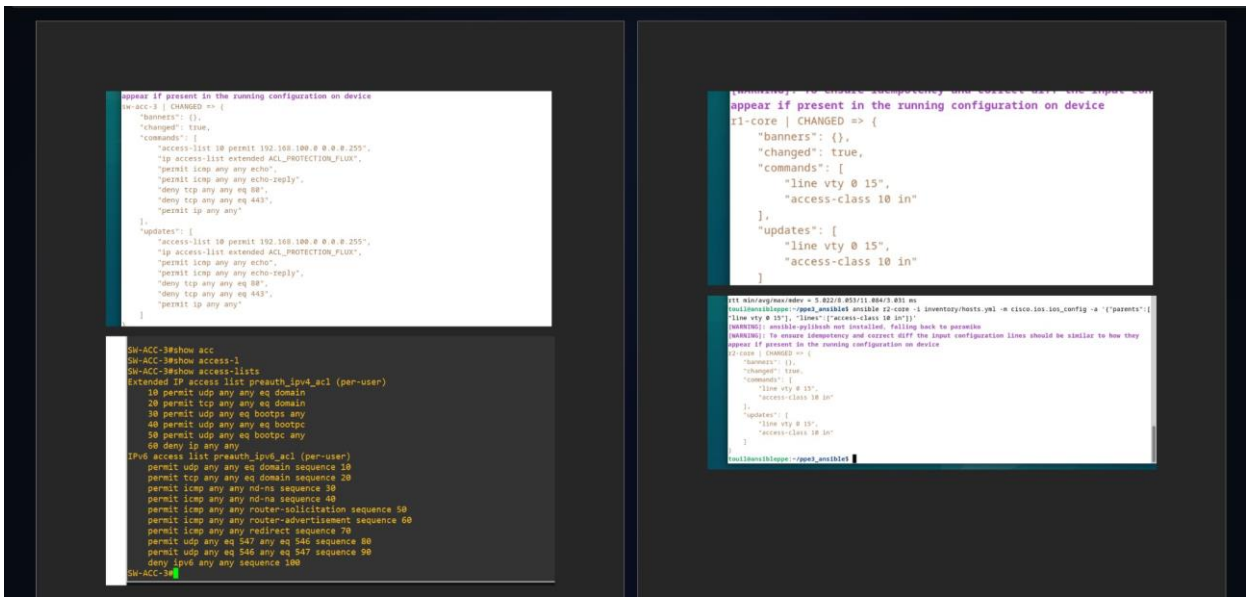

Figure 4 – Exécution simultanée sur sw-acc-3, r1-core et r2-core : configurations CHANGED appliquées (ACL et access-class VTY)

Cette seconde capture illustre la puissance du déploiement multi-équipements d'Ansible. En une seule commande, les configurations sont poussées simultanément sur sw-acc-3 (ACL de protection), r1-core et r2-core (access-class 10 sur les lignes VTY). Chaque équipement affiche le statut CHANGED avec le détail des commandes IOS appliquées. C'est exactement ce type d'opération qui prenait plusieurs heures manuellement et qui ne nécessite désormais que quelques minutes.

Tableau comparatif avant / après

Indicateur	Avant Ansible	Après Ansible
Temps de config / équipement	45 à 60 minutes	~5 minutes
Taux d'erreur	~15 %	< 1 %
Déploiement 7 équipements	~7 heures	~15 minutes
Versionnement	Aucun	Git (tracé complet)
Standardisation	Variable selon technicien	100 % identique
Rollback en cas d'erreur	Manuel et risqué	Automatisé via Git
Audit de conformité	Impossible	Playbook dédié

11. Procédure de rollback

L'un des avantages majeurs de l'intégration de Git dans le projet Ansible est la capacité de revenir rapidement à un état antérieur en cas de problème. La procédure de rollback se décompose en deux niveaux selon la criticité de l'incident.

Niveau 1 – Rollback Git (modification de playbook)

Si une modification d'un playbook ou d'une variable a introduit une erreur, on peut revenir à la version précédente grâce à Git :

```
# Consulter l'historique des commits
git log --oneline

# Revenir au commit précédent (exemple)
git revert HEAD
# ou pour revenir à un commit spécifique :
git checkout <commit_id> -- playbooks/baseline.yml

# Redéployer la version corrigée
ansible-playbook playbooks/baseline.yml
```

Niveau 2 – Rollback configuration équipement

Si une configuration erronée a déjà été déployée sur un équipement, on peut restaurer la configuration précédente en incluant une tâche de sauvegarde dans chaque playbook :

```
# Dans chaque playbook, toujours sauvegarder avant de modifier
- name: Sauvegarde configuration avant modification
  cisco.ios.ios_command:
    commands:
      - show running-config
  register: running_config

- name: Archiver la config dans un fichier local
  copy:
    content: '{{ running_config.stdout[0] }}'
    dest: './backups/{{ inventory_hostname }}_backup.cfg'

# En cas de problème, restaurer manuellement via :
# copy tftp://192.168.100.10/backups/r1-core_backup.cfg running-config
```

> **⚠ Bonne pratique**
> Il est fortement recommandé d'exécuter les playbooks avec le flag --check (mode simulation) avant tout déploiement en production. Cette option permet à Ansible de calculer les changements qui seraient appliqués sans les effectuer réellement, permettant une validation humaine préalable.

12. Guide d'utilisation

Ce guide est destiné aux techniciens de l'équipe infrastructure TechScale. Il décrit les procédures courantes d'utilisation du projet Ansible au quotidien.

Commandes essentielles

Commande	Description
`ansible all -m ping`	Tester la connectivité avec tous les équipements
`ansible-playbook playbooks/baseline.yml`	Déployer la config de base sur tous les équipements
`ansible-playbook playbooks/baseline.yml --check`	Simuler le déploiement sans appliquer
`ansible-playbook playbooks/deploy_all.yml`	Déploiement complet (toutes phases)
`ansible-playbook playbooks/baseline.yml --limit r1-core`	Déployer uniquement sur R1-CORE
`git log --oneline`	Consulter l'historique des modifications
`git add . && git commit -m 'message'`	Sauvegarder les modifications dans Git

Procédure d'ajout d'un nouvel équipement

1. Ajouter l'équipement dans inventory/hosts.yml avec son adresse IP et son groupe
2. Configurer SSH manuellement sur l'équipement (configuration initiale unique)
3. Tester la connectivité : ansible <nom_equipement> -m ping
4. Appliquer la configuration de base : ansible-playbook playbooks/baseline.yml --limit <nom>
5. Commiter la modification de l'inventaire dans Git

13. Recommandations pour la production

Le projet actuel constitue une démonstration fonctionnelle dans un environnement GNS3. Avant un déploiement en production sur les 200 équipements réels de TechScale, plusieurs améliorations sont recommandées :

Sécurité

- Remplacer l'authentification par mot de passe par une authentification par clé SSH pour les connexions Ansible
- Utiliser Ansible Vault pour chiffrer les mots de passe stockés dans les fichiers de variables
- Activer la journalisation complète des exécutions Ansible dans un SIEM
- Mettre en place une rotation régulière des credentials d'automatisation

Infrastructure

- Déployer Ansible AWX (interface web open source d'Ansible Tower) pour une gestion centralisée avec contrôle d'accès par rôles
- Mettre en place un pipeline CI/CD avec GitLab CI pour valider automatiquement les playbooks avant chaque déploiement
- Créer des roles Ansible pour factoriser les configurations répétitives et améliorer la réutilisabilité
- Configurer des inventaires dynamiques qui se synchronisent automatiquement avec la CMDB de TechScale

Monitoring

- Intégrer Ansible avec Grafana pour visualiser l'état de conformité de l'infrastructure en temps réel
- Mettre en place des playbooks de vérification planifiés (cron) pour détecter les dérives de configuration
- Configurer des alertes automatiques en cas d'échec d'un playbook en production

14. Conclusion

Ce projet PPE3 m'a permis de mettre en œuvre concrètement les principes de l'Infrastructure as Code (IaC) dans un contexte professionnel réaliste. En partant des problématiques réelles rencontrées par TechScale, j'ai conçu et déployé une solution d'automatisation complète basée sur Ansible qui répond à l'ensemble des objectifs fixés.

Les quatre phases du projet ont été réalisées avec succès. La phase 1 a permis d'établir un environnement de travail solide avec un serveur Ansible correctement configuré, une connectivité SSH établie avec tous les équipements et un dépôt Git initialisé pour le versionnement. La phase 2 a produit des playbooks fonctionnels qui appliquent une configuration de base standardisée en quelques minutes là où il fallait auparavant plus d'une heure par équipement.

La phase 3 a été la plus technique avec la mise en place du routage dynamique OSPF et de la redondance HSRP via Ansible. Ces configurations complexes, qui demandaient auparavant une expertise pointue et beaucoup de temps, sont désormais encapsulées dans des playbooks réutilisables que n'importe quel technicien de l'équipe peut exécuter. Enfin, la phase 4 a démontré la puissance du déploiement en masse avec l'exécution simultanée sur les sept équipements de l'infrastructure.

Les captures d'écran présentées dans ce dossier prouvent que les playbooks ont bien été exécutés avec succès sur les équipements Cisco dans l'environnement GNS3. Les résultats CHANGED confirment que les configurations ont été appliquées et les commandes IOS transmises aux équipements.

Sur le plan personnel, ce projet m'a permis d'approfondir mes compétences en administration réseau Cisco, en automatisation avec Ansible et en versionnement Git. Ces compétences sont directement transférables en entreprise, où l'automatisation de l'infrastructure est devenue un enjeu stratégique majeur pour maintenir la compétitivité et la fiabilité des systèmes d'information.

À terme, si cette solution était déployée sur l'ensemble du parc TechScale (200 équipements), le gain de temps estimé serait de l'ordre de 95 % sur les opérations de déploiement, avec une élimination quasi totale des erreurs de configuration. Ce projet démontre concrètement comment l'automatisation peut transformer radicalement la gestion opérationnelle d'une infrastructure réseau d'entreprise.

ℹ Bilan du projet

La solution Ansible déployée dans le cadre de ce PPE3 permet de passer d'un temps de configuration de 60 minutes à environ 5 minutes par équipement, avec un taux d'erreur réduit de 15 % à moins de 1 %. Le déploiement simultané sur 7 équipements ne prend plus que 15 minutes au lieu de 7 heures. Les configurations sont versionnées dans Git, ce qui permet un rollback rapide et sécurisé en cas d'incident.